# 1 Question 1

We first apply head $k$ specific linear transformation and obtain:

$$h_i^{(k)} = W^{(k)} z_i^{(t)}, h_j^{(k)} = W^{(k)} z_j^{(t)}.$$

We then compute unnormalized attention scores as:

$$e_{ij}^{(k)} = \text{LeakyReLU}\Big((a^{(k)})^\top \big[h_i^{(k)} \,\|\, h_j^{(k)}\big]\Big) = \text{LeakyReLU}\Big((a^{(k)})^\top \big[W^{(k)} z_i^{(t)} \,\|\, W^{(k)} z_j^{(t)}\big]\Big).$$

Now, we can compute normalized attention scores as follows:

$$\alpha_{ij}^{(k)} = \frac{\exp\left(e_{ij}^{(k)}\right)}{\sum_{l \in \mathcal{N}(i)} \exp\left(e_{il}^{(k)}\right)}.$$

Output of head $k$ is finally computed as:

$$z_i^{(t+1,k)} = \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k)} \, h_j^{(k)} = \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k)} \, W^{(k)} z_j^{(t)}.$$

.

2. We obtain the final node representation $z_i^{(t+1)}$ by concatenating all $K$ single heads:

$$z_i^{(t+1)} = \Big\|_{k=1}^{K} z_i^{(t+1,k)}.$$

The single head dimensionality is $F'_{\text{out}}$, so the dimensionality of the final node representation is $KF'_{\text{out}}$.

3. The attention vector is applied to the concatenated linear projections so its dimensionality must match the size of this concatenated vector. Weight matrix maps the input vector of dimension $F_{in}$ to an output vector of dimension $F'_{out}$. We summarize the dimensionality and number of learnable parameters for each head $k$ in the table below.

| Parameter | Dimensionality | Number of parameters |
|---|---|---|
| $W^{(k)}$ | $\mathbb{R}^{F_{\text{in}} \times F'_{\text{out}}}$ | $F_{\text{in}} F'_{\text{out}}$ |
| $a^{(k)}$ | $\mathbb{R}^{2F'_{\text{out}}}$ | $2F'_{\text{out}}$ |

Based on the table, the total parameters for a single head is $F_{\text{in}} F'_{\text{out}} + 2F'_{\text{out}}$. Since there are $K$ single heads, total number of parameters for multi-head layer is

$$K\left(F_{\text{in}} F'_{\text{out}} + 2F'_{\text{out}}\right) = KF'_{\text{out}}\left(F_{\text{in}} + 2\right).$$

# 2 Question 2

1. Based on the equations in Question 1, we have that unnormalized attention score is computes as:

$$e_{ij} = \text{LeakyReLU}\big(a^\top [\, Wx_i \,\|\, Wx_j \,]\big),$$

but since $x_i = x_j = c$, we have:

$$e_{ij} = \text{LeakyReLU}\big(a^\top [\, Wc \,\|\, Wc]\big),$$

meaning that $e_{ij}$ is not dependent on $i$ and $j$ and is the same for every pair of them. Therefore, $e_{ij}$ is a constant. If we denote $z = Wc$, the final formula for $e_{ij}$ is:

$$e_{ij} = \text{LeakyReLU}\big(a^\top [\, z \,\|\, z \,]\big).$$

The normalized attention coefficient is then derived as follows:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})} = \frac{\exp(e_{ij})}{|\mathcal{N}(i)| \, \exp(e_{ij})} = \frac{1}{|\mathcal{N}(i)|}.$$

2. The GAT layer completely loses its attention mechanism in this setting. From the derived formulas, we observe that the attention coefficients depend only on the degree of node $i$, and not on any relationship between the node and its neighbors. As a consequence, every neighbor receives the same weight regardless of its importance or context. Since the purpose of the original attention mechanism is to assign different weights based on feature interactions between nodes, we conclude that in this case the layer can no longer behave as an attention mechanism.
The layer degenerates into the mean aggregation GNN propagation rule. Indeed, the representation for node $i$ becomes:

$$x_i^{(t+1)} = \sum_{j \in \mathcal{N}(i)} \alpha_{ij} \, W x_j^{(t)} = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} W x_j^{(t)},$$

which shows that the layer simply computes the average of the neighbor features. This is the standard mean aggregation rule.

3. We recall that the Karate network contains two mostly separated groups, with many edges within each group and a fewer edges between them. Our degenerated GAT layer can exploit the described structure. As shown, the node representations are updated by averaging the neighbor features, which means that the nodes from the same group will receive similar updates, while the nodes across different group will receive different updates. Over multiple layers and propagation, the nodes within the same group will be kept together, which will, over time, lead to two clusters that are more easily distinguishable than with random guessing.

# 3 Question 3

We can follow the approach from [1] and [2]. Let $c$ denote a conditioning vector. To introduce conditioning, we modify both the encoder and the decoder:

- Encoder: We concatenate $c$ to every node's feature vector before the global pooling layer. The variational posterior becomes:

$$q_\phi(\mathbf{z}|G, c) = q_\phi(z|X, A, c) = \mathcal{N}\big(z|\mu(X, A, c), \mathrm{diag}(\sigma^2(X, A, c))\big).$$

- Decoder: We feed the decoder with the concatenation of $z$ and $c$. Its generative distribution becomes:

$$p_\theta(G|z, c).$$

  In the case of one-shot decoder, we can reconstruct the adjacency matrix as follows:

$$\hat{A} = \mathrm{MLP}([z \,\|\, c]).$$

# 4 Question 4

If we were to sample directly from a distribution whose parameters VAE predicts, i.e. $\mathcal{N}(\mu, \sigma^2 I)$, the sampling would not be differentiable w.r.t. $\mu$ and $\sigma$. Since backpropagation updates parameters based on the computed gradients through the computational graph, the encoder parameters could not be updated and the training would therefore fail. With reparameterization, we separate the randomness in $\epsilon$ from the deterministic $\mu$ and $\sigma^2$. In this form, $z$ becomes differentiable w.r.t $\mu$ and $\sigma$:

$$\frac{\partial z}{\partial \mu} = 1, \qquad \frac{\partial z}{\partial \sigma} = \epsilon.$$

We have shown that sampling from $\mathcal{N}(\mu, \sigma^2 I)$ breaks the differentiability and prevents training with standard backpropagation. In general, we can still train our model using score functions to estimate gradients, as they do not require differentiation. However, this approach is less stable and efficient, so we prefer to use the reparameterization trick.

# References

[1] Martin Simonovsky and Nikos Komodakis. Graphvae: Towards generation of small graphs using variational autoencoders. In *Artificial Neural Networks and Machine Learning – ICANN 2018 (Lecture Notes in Computer Science)*, pages 412–422. Springer, 2018.

[2] Kihyuk Sohn, Honglak Lee, and Xinchen Yan. Learning structured output representation using deep conditional generative models. In *Advances in Neural Information Processing Systems 28 (NeurIPS 2015)*, pages 3483–3491, 2015.